

南开大学

汇编语言与逆向技术课程实验报告

Lab8: Reverse Engineering Exercises



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

目录

1 实验时间	2
2 实验目的	2
3 实验环境	2
4 反汇编代码 Binary Ninja 视窗	2
5 task1 逆向分析	3
5.1 程序流程图	3
5.2 详细过程	3
5.2.1 字符串长度验证	3
5.2.2 逐个字符条件验证	4
5.3 代码求解	6
5.4 数据结构	7
6 task2 逆向分析	7
6.1 程序流程图	7
6.2 详细过程	7
6.3 代码求解	8
6.4 数据结构	9
7 实验结果	9
8 附录：用于求解的完整 python 代码	9

1 实验时间

第 15 周星期一 9-10 节

2 实验目的

1. 熟悉静态反汇编工具 Binary Ninja;
2. 熟悉反汇编代码的逆向分析过程;
3. 掌握反汇编语言中的数学计算、数据结构、条件判断、分支结构的识别和逆向分析

3 实验环境

Binary Ninja、PyCharm

4 反汇编代码 Binary Ninja 视窗

使用 Binary Ninja 打开待分析的 PE 文件 task1.exe 和 task2.exe

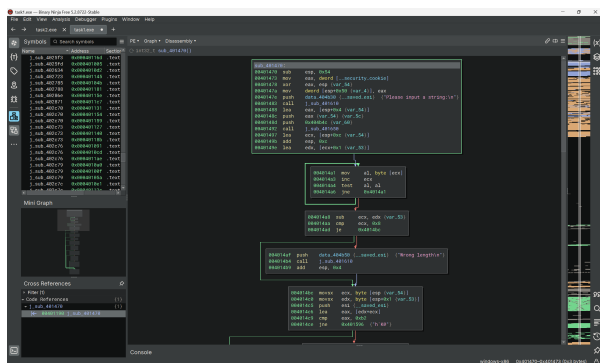


图 4.1: task1 反汇编代码图形化视窗

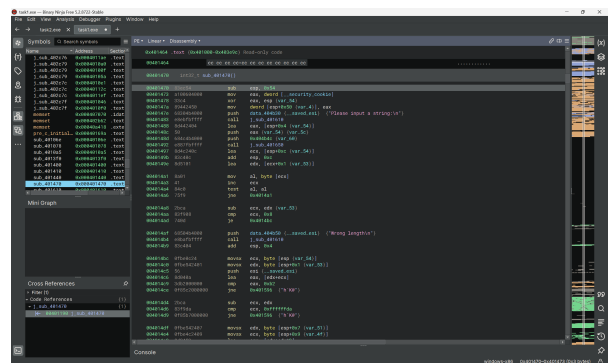


图 4.2: task1 反汇编代码

图 4.3: task1

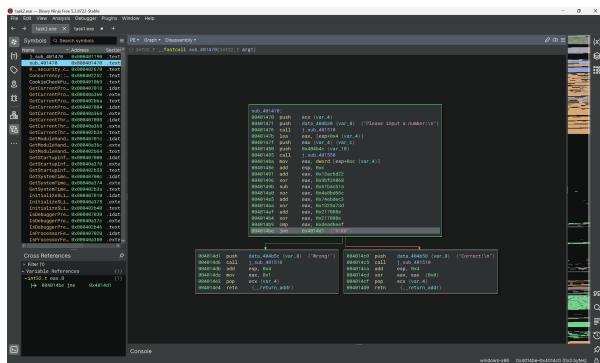


图 4.4: task2 反汇编代码图形化视窗

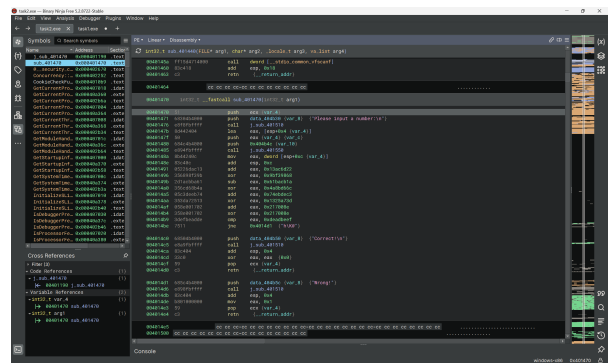
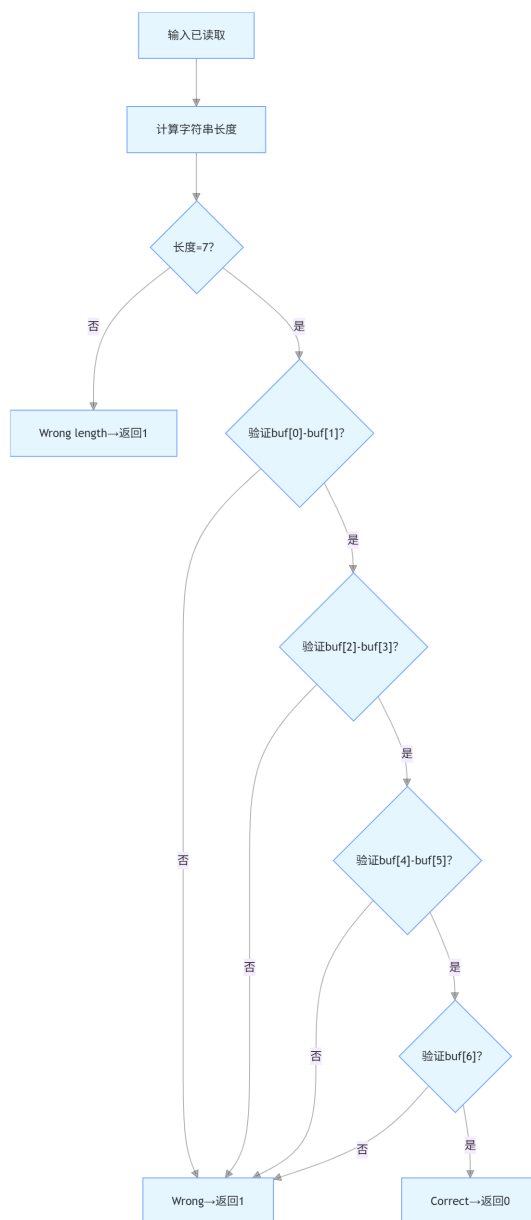


图 4.5: task2 反汇编代码

图 4.6: task2

5 task1 逆向分析

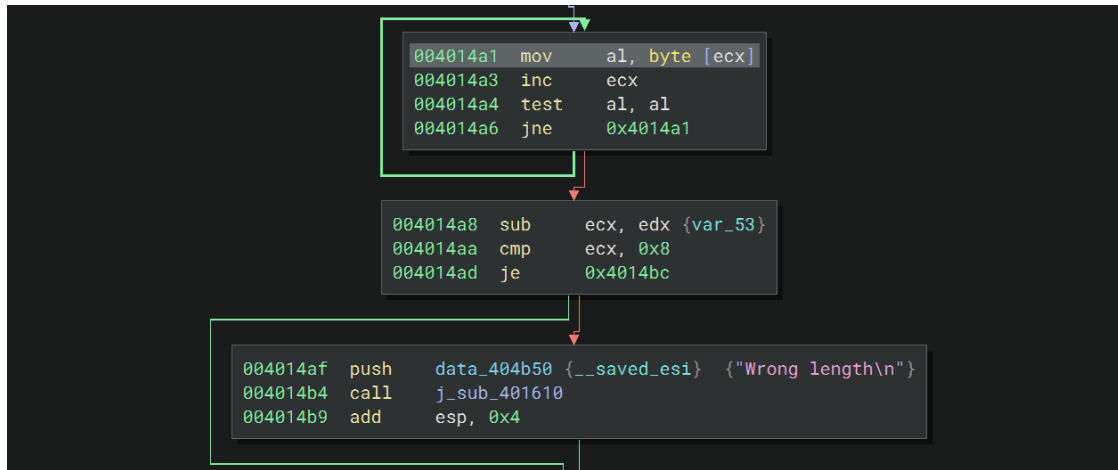
5.1 程序流程图



5.2 详细过程

5.2.1 字符串长度验证

首先进入一个循环，遍历字符串定位字符串末尾的 `\0`。以地址差 = 8 为判定条件，校验输入字符串的有效长度。如果输入字符串为 8 个字符长，通过进入下一个条件验证；如果不等于 8，打印错误信息 "Wrong length"。



5.2.2 逐个字符条件验证

假设用户输入为 8 个字符的字符串，记 $s[0]$ 到 $s[7]$ ，验证过程包含以下 8 个数学条件：

1. 条件一：将第一个字符 $s[0]$ 存入 ecx ，第二个字符 $s[1]$ 存入 edx ，判断两数之和是否等于 $0xB2$ （十进制 178），若不相等则跳转到错误处理。

```

1      lea    eax, [edx+ecx]          ; EAX = s[1] + s[0]
2      cmp    eax, 0xb2              ; 比较和是否等于178 (0xB2)
3      jne    0x401596 {"h`K@"}      ; 如果不相等，跳转到错误处理

```

2. 条件二：在条件一的基础上，判断 $s[0] - s[1]$ 是否等于 $0xFFFFFDA$ （十进制-38），若不相等则验证失败。

```

1      sub    ecx, edx              ; ECX = s[0] - s[1]
2      cmp    ecx, 0xfffffda        ;
                                     比较差值是否等于-38 (0xFFFFFDA为-38的补码表示)
3      jne    0x401596 {"h`K@"}      ; 如果不相等，跳转到错误处理

```

3. 条件三：将第四个字符 $s[3]$ 存入 edx ，第六个字符 $s[5]$ 存入 ecx ，判断表达式 $3*s[3] + 2*s[5]$ 是否等于 $0x21D$ （十进制 541）。

```

1      lea    eax, [edx+edx*2]       ; EAX = s[3] * 3
2      lea    eax, [eax+ecx*2]       ; EAX = (s[3] * 3) + (s[5] * 2)
3      cmp    eax, 0x21d            ; 比较是否等于541 (0x21D)
4      jne    0x401596 {"h`K@"}      ; 如果不相等，跳转到错误处理

```

4. 条件四：在条件三的基础上，判断 $s[5] - 8*s[3]$ 是否等于 $0xFFFFD3C$ （十进制-708）。

```

1      lea    eax, [edx*8]           ; EAX = s[3] * 8
2      sub    ecx, eax              ; ECX = s[5] - (s[3] * 8)
3      cmp    ecx, 0xfffffd3c        ;
                                     比较是否等于-708 (0xFFFFD3C为-708的补码表示)

```

4 jne 0x401596 {"h`K@"} ; 如果不相等, 跳转到错误处理

5. 条件五: 将第五个字符 s[4] 存入 esi, 第八个字符 s[7] 存入 edx, 判断表达式 $125*s[4] + 18*s[7]$ 是否等于 0x2AD9 (十进制 10969)。

```

1      imul    ecx, esi, 0x7d          ; ECX = s[4] * 125 (0x7D = 125)
2      lea     eax, [edx+edx*8]        ; EAX = s[7] * 9
3      lea     eax, [ecx+eax*2]        ; EAX = (s[4] * 125) + (s[7] * 18)
4      cmp     eax, 0x2ad9             ; 比较是否等于10969 (0x2AD9)
5      jne     0x401596 {"h`K@"}      ; 如果不相等, 跳转到错误处理

```

6. 条件六: 在条件五的基础上, 判断表达式 $6*s[7] - 33*s[4]$ 是否等于 0xFFFFF613 (十进制-2541)。

```

1      mov     eax, esi                ; EAX = s[4] (第五个字符)
2      lea     ecx, [edx+edx*2]        ; ECX = s[7] * 3 (第八个字符的3倍)
3      shl     eax, 0x5                ; EAX = s[4] << 5 = s[4] * 32
4      add     ecx, ecx                ; ECX = s[7] * 6
5      add     eax, esi                ; EAX = (s[4] * 32) + s[4] = s[4] * 33
6      sub     ecx, eax                ; ECX = (s[7] * 6) - (s[4] * 33)
7      cmp     ecx, 0xfffff613         ;
      比较是否等于-2541 (0xFFFFF613为-2541的补码表示)
8      jne     0x401596 {"h`K@"}      ; 如果不相等, 跳转到错误处理

```

7. 条件七: 将第七个字符 s[6] 存入 edx, 第三个字符 s[2] 存入 esi, 判断表达式 $10*s[6] - 3*s[2]$ 是否等于 0x351 (十进制 849)。

```

1      lea     ecx, [edx+edx*4]        ; ECX = s[6] * 5
2      add     ecx, ecx                ; ECX = s[6] * 10
3      lea     eax, [esi+esi*2]        ; EAX = s[2] * 3
4      sub     ecx, eax                ; ECX = (s[6] * 10) - (s[2] * 3)
5      cmp     ecx, 0x351              ; 比较是否等于849 (0x351)
6      jne     0x401596 {"h`K@"}      ; 如果不相等, 跳转到错误处理

```

8. 条件八: 在条件七的基础上, 判断表达式 $62*s[2] - 7*s[6]$ 是否等于 0x1460 (十进制 5216)。

```

1      mov     ecx, esi                ; ECX = s[2] (第三个字符)
2      lea     eax, [edx*8]            ; EAX = s[6] * 8 (第七个字符的8倍)
3      shl     ecx, 0x5                ; ECX = s[2] << 5 = s[2] * 32
4      sub     eax, edx                ; EAX = (s[6] * 8) - s[6] = s[6] * 7
5      sub     ecx, esi                ; ECX = (s[2] * 32) - s[2] = s[2] * 31
6      add     ecx, ecx                ; ECX = s[2] * 62
7      sub     ecx, eax                ; ECX = (s[2] * 62) - (s[6] * 7)
8      cmp     ecx, 0x1460             ; 比较是否等于5216 (0x1460)

```

9

jne 0x401596 {"h`K@"} ; 如果不相等, 跳转到错误处理



如果 8 个条件全部验证通过, 输出 “Correct”; 否则输出 “Wrong”。

5.3 代码求解

依据上述分析, 可以编写 python 代码, 运行得到结果。

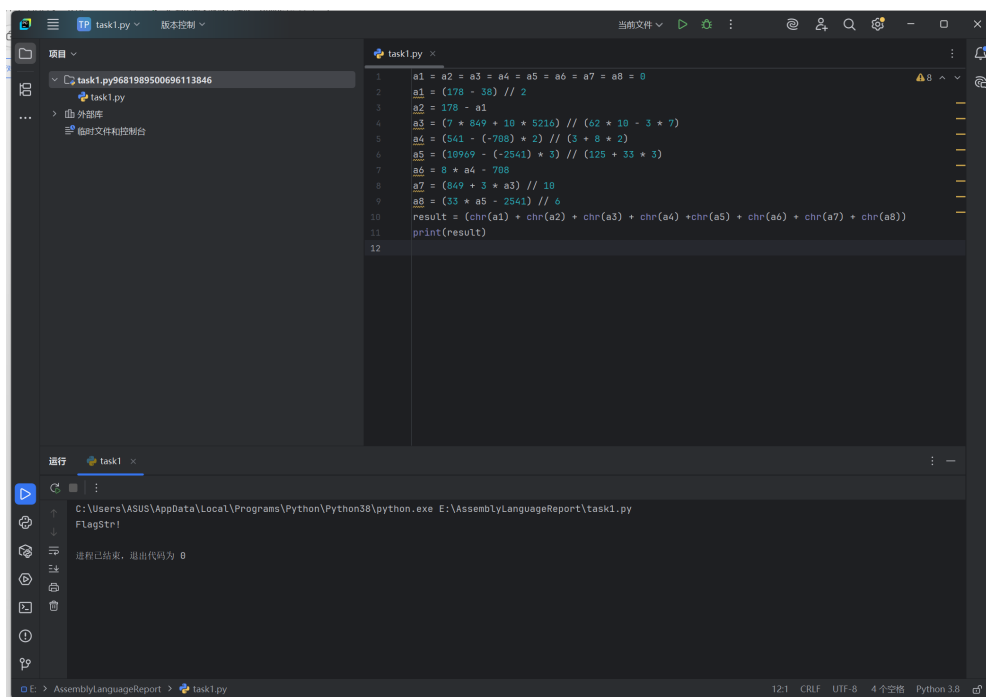


图 5.7: task1 python 代码运行截图

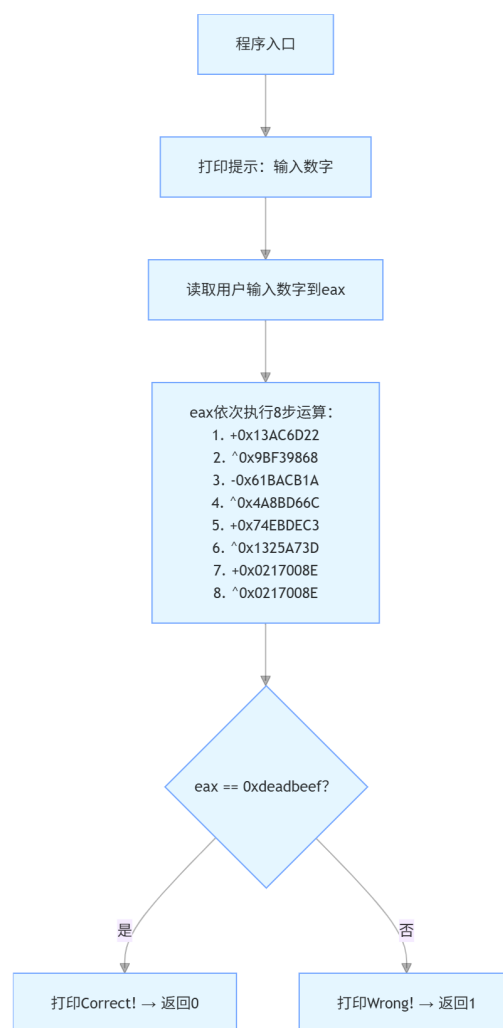
结果为: FlagStr!

5.4 数据结构

这段汇编代码无自定义复杂数据结构，核心数据结构围绕栈 Stack 展开，所有数据操作均基于栈、通用寄存器和只读字符串常量。通过栈完成局部数据存储、函数参数传递、栈保护、寄存器备份等。

6 task2 逆向分析

6.1 程序流程图



6.2 详细过程

task2 是一个数值计算验证。通过 scanf 读取一个整数到 eax，结合了加法、减法和异或运算，验证该数是否符合条件

1. 将 eax 的值加上 0x13AC6D22

2. 将上一步的结果与 0x9BF39868 进行异或 (XOR) 操作
3. 从上一步的结果中减去 0x61BACB1A
4. 将上一步的结果与 0x4A8BD66C 进行异或 (XOR) 操作
5. 将上一步的结果加上 0x74EBDEC3
6. 将上一步的结果与 0x1325A73D 进行异或 (XOR) 操作
7. 将上一步的结果加上 0x0217008E
8. 将上一步的结果与 0x0217008E 进行异或 (XOR) 操作

```

1  add    eax, 0x13ac6d22      ; 第一步: 输入值加上0x13AC6D22
2  xor    eax, 0x9bf39868     ; 第二步: 结果与0x9BF39868异或
3  sub    eax, 0x61bacb1a     ; 第三步: 结果减去0x61BACB1A
4  xor    eax, 0x4a8bd66c     ; 第四步: 结果与0x4A8BD66C异或
5  add    eax, 0x74ebdec3     ; 第五步: 结果加上0x74EBDEC3
6  xor    eax, 0x1325a73d     ; 第六步: 结果与0x1325A73D异或
7  add    eax, 0x217008e      ; 第七步: 结果加上0x217008E
8  xor    eax, 0x217008e      ; 第八步: 结果与0x217008E异或
9  cmp    eax, 0xdeadbeef     ; 比较最终结果是否等于0xDEADBEEF
10 jne    0x4014d1 {"h\K@"}   ; 如果不相等, 跳转到错误处理

```

6.3 代码求解

编写 python 代码，运行得到结果。

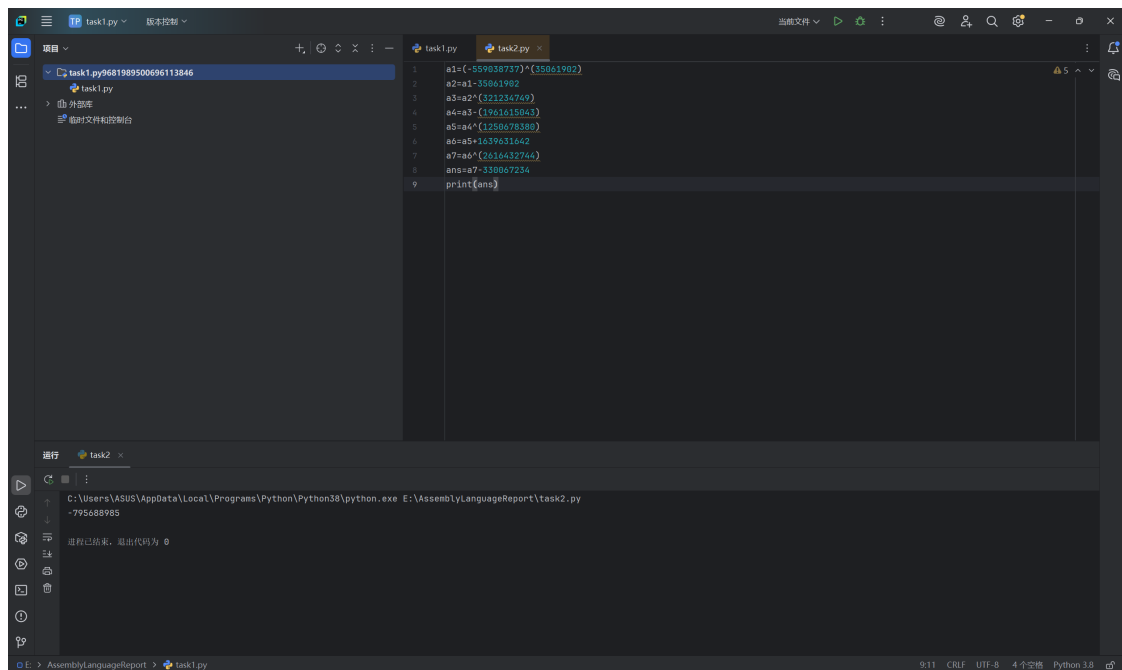


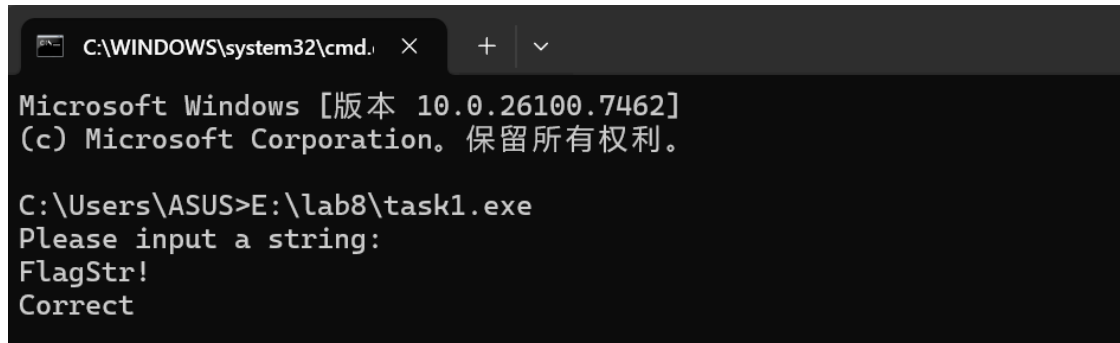
图 6.8: task2 python 代码运行截图

结果为: -795688985

6.4 数据结构

这段汇编代码无自定义复杂数据结构，核心数据结构围绕栈 Stack 展开，所有数据操作均基于栈和通用寄存器完成。如 `[esp+0xc]`，输入的数字暂存于栈缓冲区。

7 实验结果

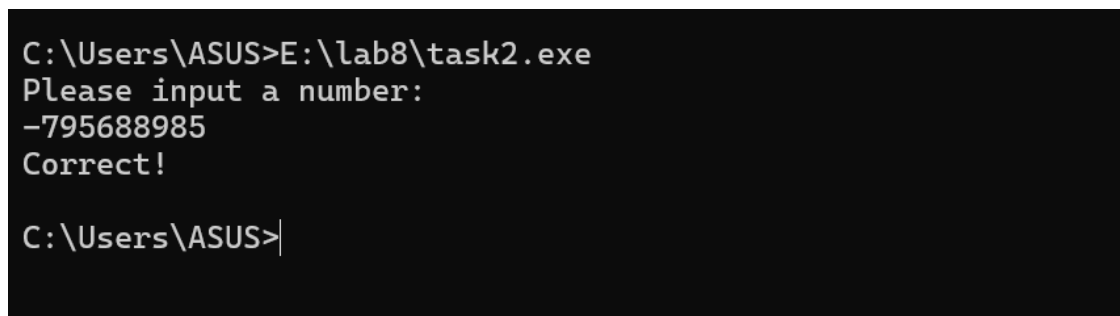


```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [版本 10.0.26100.7462]
(c) Microsoft Corporation。保留所有权利。

C:\Users\ASUS>E:\lab8\task1.exe
Please input a string:
FlagStr!
Correct
```

图 7.9: task1 实验结果截图

得到了 “Correct”。



```
C:\Users\ASUS>E:\lab8\task2.exe
Please input a number:
-795688985
Correct!

C:\Users\ASUS>|
```

图 7.10: task2 实验结果截图

得到了”Correct!”。

8 附录：用于求解的完整 python 代码

task1 代码

```
1 def main():
2     str_input = input().strip()
3     str_len = len(str_input)
4     num1 = 0
5     num2 = 0
6     num3 = 0
```

```
7     num4 = 0
8     if str_len >= 5:
9         num1 = ord(str_input[1]) + ord(str_input[3]) + ord(str_input[4])
10        num2 = ((~0xBADF000D & 0xFFFFFFFF) - 0x18) & 0xFFFFFFFF
11        if str_len >= 3:
12            num3 = (ord(str_input[0]) * ord(str_input[2])) * 0x0c48
13        for i in range(str_len):
14            num4 += ord(str_input[i])
15        num4 = ((num4 + 0x7b) ^ 0x31337) & 0xFFFFFFFF
16        print(f"{num1}-{num2}-{num3}-{num4}")
17
18 if __name__ == "__main__":
19     main()
```

task2 代码

```
1 a1=(-559038737)^(35061902)
2 a2=a1-35061902
3 a3=a2^(321234749)
4 a4=a3-(1961615043)
5 a5=a4^(1250678380)
6 a6=a5+1639631642
7 a7=a6^(2616432744)
8 ans=a7-330067234
9 print(ans)
```